

GPU kernels · Triton · agentic

Agentic-KernelBench

A harder, fairer, **agentic** redesign of KernelBench — can a model **iteratively write a Triton kernel that beats** **torch.compile** on the architectures people actually serve?

10 modern families · ~132 tasks · pass@2 on GPT-5.5 / Opus-4.8 / Gemini-3.5-flash

§1 · CHOSEN DOMAIN

GPU kernel generation — and why it's a meaningful extension

Domain: **write a fast GPU kernel** for a given PyTorch module. We extend **KernelBench** (Stanford, arXiv:2502.10517) into an **agentic**, modern-ops, beat-the-compiler benchmark.

- **High real value** — kernels are the bottleneck of LLM training/serving cost.
- **Frontier-relevant** — writing fast GPU code is exactly where models are still weak.

Verification is unambiguous

Ask the model to **rewrite the PyTorch reference into a custom Triton kernel**; the verifier checks **(1) the output matches** the reference and **(2) how much the runtime improves** over `torch.compile`.

→ correctness + a numeric speedup = a clean, objective pass/fail.

§2 · BENCHMARK ANALYSIS

The original KernelBench is now too easy for frontier models

Weakness	Why it makes tasks easy
Single-shot	no compile/profile feedback — measures recall, not engineering like a human
Classic ops	conv/matmul/elementwise are memorized; flash-attn is known
Weak baseline (eager)	beating eager is easy; torch.compile(max-autotune) runs after compiler optimizations, so it's faster

After researching where models still fail, the hard pockets are **modern, quantized, fused** ops — so that's where we build.

§3 · REDESIGN & TASK DESIGN

Four main improvements over the original KernelBench

① Agentic

Harbor container; the model compiles, runs, reads errors, resubmits over **4 turns**. Harder interaction forces harder tasks.

② Modern SOTA architectures

NSA, MLA, MoE, quant, fused — more complex and state-of-the-art than KernelBench's classic ops.

③ Strong baseline

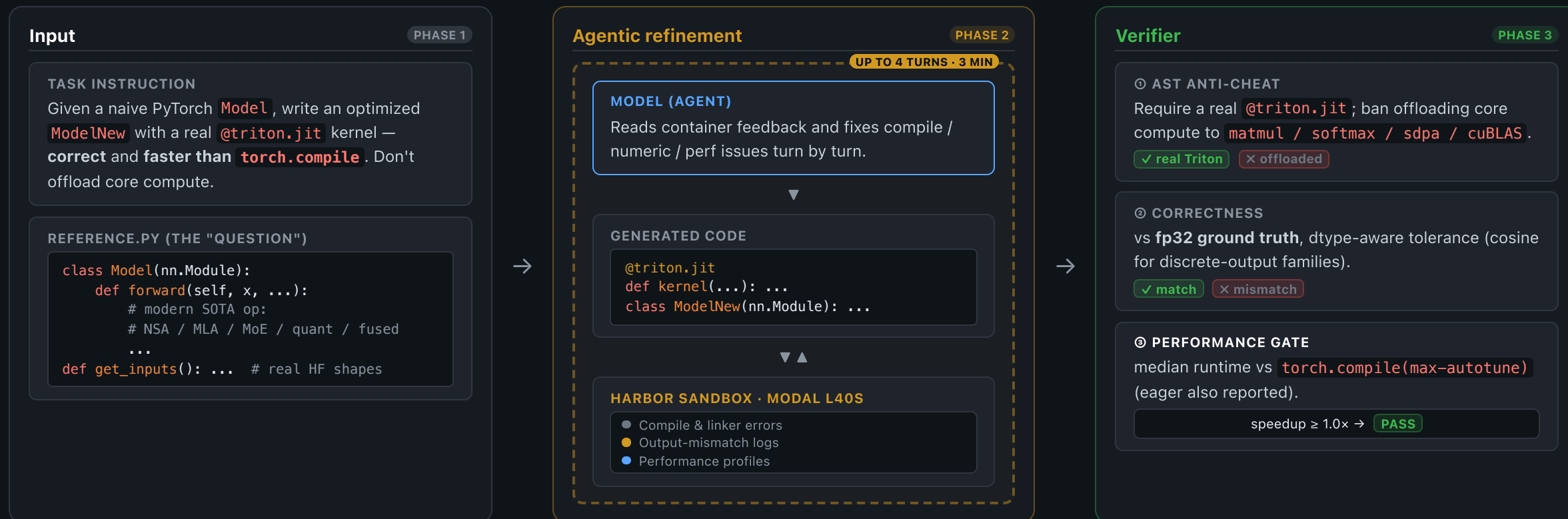
Gate = beat `torch.compile(max-autotune)` ; eager speedup also reported. Correct-but-slow fails.

④ Anti-cheat (pytorch2triton)

AST bans offloading core compute to `torch.matmul /softmax/sdpa/cuBLAS`; requires a real `@triton.jit`.

§3 · REDESIGN & TASK DESIGN

How it works — the agentic verify pipeline



§3 · REDESIGN & TASK DESIGN

10 modern families → ~132 tasks

NSA

DeepSeek block-sparse (2025)

MLA decode

DeepSeek-V2 absorb

MoE top-2/6/8

Mixtral dispatch

W4A16

packed **asymmetric** int4

W8A8 SmoothQuant

int8 requant

FP8 microscaling

block-scaled GEMM

fused SiLU+FP8

vLLM deep_gemm

fused quant-MoE

dequant+grouped GEMM

multi-LoRA qGEMM

batched adapters

SSM scan

Mamba selective scan

All from **real architectures** (DeepSeek, Mixtral, Llama, Qwen, Mamba) — well-distributed across the modern stack.

§3 · REDESIGN & TASK DESIGN

Difficulty = strong baseline + a tricky numerical contract

Fusion alone isn't hard. The gap is the **exact dequant / scaling spec** the kernel must match, e.g.:

- **W4A16:** `w = (unpack4(qweight) - zero[g]) × scale[g]` — unpack nibbles in the right order, per-group **asymmetric** zero-point
- **FP8 block-scale:** `s = amax(block)/448; q = round_e4m3(x/s); out = q·s` per 128-group
- **NSA select:** score KV blocks by mean-key, keep the top-k **causal** blocks, attend only within them

Difficulty tracks baseline strength

We **drop weak-baseline ops** (vanilla attention, naive SSM) frontier models clear in one shot, and keep families where `torch.compile` /cuBLAS leave real fusion headroom.

§3 · REDESIGN & TASK DESIGN

Infra, resource budget & how it scales

Workflow

Harbor (agentic env) · **Modal** (GPU verify, locked L40S) · **OpenRouter** (model calls).

Resource budget

4 turns (AMD's arena uses 5) + a **~3 min** agent timeout → bounds compute/\$ (turns & time).

Scale from one hard task

Hard template → neighbors via **parameter grids** over real HF configs (model, size, gate/down layer, group size, #experts, dtype). One template → 15–30 tasks.

quant 24 · moe 20 · fp8/mla/w8a8 18 each · ssm 10 · fused/lora 6–9 · nsa 4

Oracles for all 10 families (incl. the hardest) — proof every task is solvable.

§4 · MODEL RESULTS

pass@1 & pass@2 — 3 frontier models, agentic, 2 samples/task

Model	pass@1	pass@2
GPT-5.5	40% (8/20)	50% (5/10)
Opus-4.8	25% (5/20)	40% (4/10)
Gemini-3.5-flash	5% (1/20)	10% (1/10)

- GPT/Opus squarely in the target **10–60% band**; clean gradient GPT > Opus > flash.
- Reproducible from the logs via `results/compute_passk.py`.

§4 · MODEL RESULTS

Per-family difficulty (pass@2) — gradient + hard tail

Family	GPT	Opus	flash
fp8_microscale	✓	×	✓
moe_top2	✓	✓	×
ssm_scan	✓	✓	×
mha_decode	×	✓	×
w8a8_smoothquant	✓	×	×
fused_silu_fp8 (cosine)	×	✓	×
nsa_block_sparse (n2048)	✓	×	×
quant_w4a16_pa	×	×	×
fused_qmoe	×	×	×
multi_lora_qgemm	×	×	×

3 families unsolved by *any* model = hard tail — yet **all 10 have a passing oracle**, so all are provably solvable.

§4 · MODEL RESULTS

Finding: capable models turn the feedback loop into fixes

GPT-5.5 — iteration repairs

Reads compiler/profiler feedback and fixes errors over turns — e.g. it lands a passing **1.18×** NSA kernel by iterating through compile errors. That's the value the agentic loop adds.

flash — weakest of the three

10% pass@2 — it rarely turns the compiler/profiler feedback into a correct, faster kernel.

The agentic loop pays off when a model can act on feedback — fix the compile error first, then make it fast.

§5 · QC METHODOLOGY

How I checked task quality: unambiguous, reliable, fair

- **Unambiguous** — pass/fail is fully objective: the output **matches the reference and beats** `torch.compile` . No judgment calls.
- **Reliable verifier** — every family ships a known-good **oracle that passes**, so the verifier provably accepts correct kernels (no false-negatives) and each task is solvable.
- **Fair** — every model gets the same instructions and turn/time budget, and every task has a verified-correct oracle, so all are solvable. Difficulty comes from the operator, not tricks — the tasks are based on real architectures and systems.

Difficulty comes from the operator — every task is oracle-verified solvable.

§6 · TRAINING-SIGNAL VALUE

Why this data is valuable to train on

- **Dense, verifiable reward** — the pass gate is **speedup over `torch.compile(max-autotune)`** (eager also reported); a continuous speedup is a natural RL reward, far richer than pass/fail.
- **Un-gameable** — AST anti-cheat + "beat compile" gate; the model must do real work, not call cuBLAS.
- **In the right difficulty band** — 10–60%: dense learning signal, neither saturated nor impossible.
- **In-distribution & real** — every task is a real deployment shape (real HF configs).
- **Agentic trajectories** — multi-turn compile→fix→profile traces are exactly the behavior we want to teach.

From one hard task to 100K

The loop

- ① **Find a hard task** — e.g. a fused vLLM kernel; inspired by the takeaways of current benchmarks (what's still unsolved).
- ② **Probe single-shot** → confirm it's hard enough that some attempts fail (not trivially solved).
- ③ **Scale the params** → a grid of neighbors.
- ④ **Evaluate agentially** on Modal.

Scaling params (real HF configs)

open-source model family · size · layer role (gate / down) · group size · #experts · dtype · sequence length

Volume vs effort

Scale	How	GPU eval
100s	grid over HF configs	~\$30–60
1K	+templates + denser grids	~\$300–600
10K	mine real HF model configs + auto-filter	~\$3–6K
100K	crawler + LLM-assisted synthesis	~\$30–60K

Verifier written once & shared; difficulty inherited by every grid point, estimated by sampling — not exhaustive runs.

RELATED WORK · WHY OURS STANDS OUT

Where ours sits in a brand-new frontier

Benchmark	Interaction	Task	Baseline
KernelBench ('24)	single-shot	PyTorch→CUDA/Triton	eager
KernelBench-X ('26)	single-shot	→Triton (176 tasks)	eager
Robust-KBench (Sakana '25)	agentic	PyTorch→ CUDA	eager
AMD AgentKernelArena ('26)	agentic (5-turn)	Triton→Triton	given kernel
Ours	agentic (4-turn)	PyTorch→Triton	torch.compile

- Agentic kernel-gen is a **2025–26 frontier** — closest prior work: Robust-KBench (→CUDA) & AMD (→Triton-from-Triton).
- **What's distinctive about ours:** agentic **pytorch→Triton from scratch** · beat **torch.compile** · modern SOTA ops · **oracle-proven solvable**.

REFERENCES

References & inspirations

Work	Paper	Code
KernelBench (Stanford)	arXiv:2502.10517	github.com/ScalingIntelligence/KernelBench
Robust-KBench (Sakana AI)	arXiv:2509.14279	github.com/SakanaAI/robust-kbench
KernelBench-X	arXiv:2605.04956 (May '26)	github.com/BonnieW05/KernelBenchX
AMD AgentKernelArena	arXiv:2605.16819 (May '26)	github.com/AMD-AGI/AgentKernelArena
Survey — Automated GPU kernels	simonguo.tech/blog/2025-10-automated-gpu-kernels.html	

Inspirations: vLLM golden kernels (deep_gemm→fused_silu, fused_moe→fused_qmoe) · DeepSeek NSA / MLA · quantization as a hard-task family.

FUTURE WORK

Takeaways & what's next

Takeaways

- A **harder, fairer, agentic** KernelBench: beat `torch.compile` on modern kernels, un-gameable.
- Frontier models in the **10–60%** band; **10/10 families proven solvable** by oracles.

Future work

- ① **Extend to more modern architectures and their latest tricky numerical contracts** (longer context, larger turn budgets).
- ② **Train with it** — use the speedup-over-`torch.compile` as a dense **RL reward**.

Thank you! I really enjoyed doing this, and would love more time to extend it to other modern architectures.